Induction heads in a browser tab: a hand-written WebGPU trainer, and what the repeat rule and weight tying do to onset

Abhay Mettu
Independent

https://github.com/abhaymettu/induction-tab

https://abhaymettu.github.io/induction-tab/

Abstract

Induction heads form abruptly during training, and a reader of that literature usually meets the transition as a plot from a finished run. This paper trains a two-layer transformer through the transition in a browser tab, on the viewer's GPU. On an M4 Pro the median to the event is 4.9 s with drawing off. With drawing on, beside another GPU job, the page reads 8.7 s, and 5.8 s on each seed's fastest round (Section 2). The trainer is hand-written WGSL for the forward pass, the backward pass and AdamW. Against PyTorch it reaches the same event and anchor step on 64 of 64 seeds at each of two model sizes, with runs compared up to the event. We then apply Gunner Howe's event and anchor, with Olsson et al.'s head scores, to a smaller study model, under three plans committed to git, with 8 to 64 seeds per arm. Section 9 lists what ran before a plan and where the plans were not followed. First, the repeat rule is the largest single-factor contrast we can bound in steps. With everything else fixed, tiled repeats fire on 16 of 16 seeds at a median of step 1025, and Howe's clone rule on 2 of 16 by step 8000. The period range changes direction between the two rules. Second, on random repeats, tying weights delayed the event at equal depth, with time ratios of 1.55, 2.31 and 3.27 by a censored fit and 1.52, 2.17 and 2.81 by Kaplan-Meier medians at depths 2, 3 and 4 and lr 3e-3. The delay holds at both learning rates and at depth 12, with no crossover, and we did not separate effective step size from structure. Third, the anchor fraction fell below Howe's envelope on 3 arms. Eval spacing explains part of one miss, and the deepest tied arms stay outside at 0.495 and 0.161.

1 Introduction

This paper trains a transformer through the induction transition in a browser tab, on the viewer's own GPU, while the viewer watches. For the first few hundred steps the model has learned nothing. Then three things happen close together. The second copy of a repeated random sequence lights up as the model starts to predict it, the attention maps draw the two lines that Olsson et al. [2] described as the induction circuit, and the copy advantage crosses 2 nats. On an M4 Pro in Chrome, seeds 1 to 5 reach that event at steps 500, 475, 600, 425, and 475, which is 5.2, 4.9, 6.2, 4.4, and 4.9 s with drawing off. The viewer can then type a string that repeats and watch the model continue it, or silence a head and watch copying stop.

The point of doing this in a tab is that the transition becomes something a person can watch and poke. A run is short enough to repeat with another seed, a head can be silenced as soon as its line appears, and nothing has to be installed. Pope and Tedre's Little Language Machine [13] reports a shift in how students explain language models after training one in the browser. We ran no study of learners and make no claim about teaching.

Other browser demos train induction heads live, including TheoremPath's lab [15] and gama's Induction [16], both on the CPU or without a stated GPU backend. This one runs on the viewer's GPU in hand-written WGSL, keeps MLPs and LayerNorm, and streams previous-token and induction scores with head ablation on one page. We did not find that combination within our search coverage. TabGPT [17] and webgpu-llm [18] also write their training kernels by hand in WGSL, and Vin Howe's Piston and Sequence Toy [14] train sequence models in the browser on WebGPU through an automatic differentiation library.

Beyond the demo, this paper reports the following. Section 2 checks the trainer against PyTorch and times it. Section 3 says what the instruments, from Olsson et al. [2] and Gunner Howe [1], do and do not measure. Section 4 reports one draw at Howe's own configuration. Sections 5 and 6 use the instruments on a small study model to ask what sets the onset of the event and whether Howe's forecast holds there. Section 7 lists exploratory reads, and Section 9 reports what failed. Nothing here is a statement about large models. The study model has width 64 and context 64.

Table 1: The WGSL trainer against PyTorch, seed by seed, up to the event. The anchor and the event are defined in Section 3.

	Tab model (d=128, T=128)	Study model (d=64, T=64)
Seeds	64	64
Same event step	64	64
Same anchor step	64	64
Median event step, each stack	475	575
Evals compared	1336	1539
Evals with copy advantage within 1e-5	1328	1539
Worst copy advantage difference (nats)	6.5e-5	3.2e-6
Worst training loss difference (nats)	1.4e-5	8.1e-6

2 The trainer

The page uses no ML library. The forward pass, the backward pass and AdamW are hand-written WGSL compute shaders in 32-bit floats, and one training step is 65 dispatches. The model follows Howe's block layout: pre-LayerNorm, attention and a GELU MLP, learned absolute positions, a final LayerNorm and an untied unembedding. The tab's model has vocabulary V=96, context T=128, width d=128, H=4 heads, two layers, an MLP ratio of 4, batch 16 and lr 3e-3, for 435456 parameters, with LayerNorm that has no learned gains. AdamW uses betas of 0.9 and 0.95, eps 1e-8, weight decay 0.01 and a 100-step linear warmup, as Howe's trainer does. Training data is uniform random tokens tiled at a random period in [4, 64), so the one thing to learn is that characters repeat.

Gradients. A parity tool feeds the same weights and batches to PyTorch autograd and to the trainer for 20 steps and compares every parameter tensor, on 5 configurations including the tab's own. In the stored run on the M4 Pro, the worst per-tensor relative L2 difference is 1.1e-6 for a gradient and 5.0e-7 for a parameter, and the largest absolute loss difference is 7.2e-6. These move slightly between runs. A stored run of 2 configurations on SwiftShader reads 5.6e-7 for a gradient.

Whole runs. Table 1 compares complete runs on 64 seeds at two model sizes. One stack is WGSL on Metal and the other is PyTorch on a CPU. Both start from the same weights and draw the same batches, because the parameter layout, the initializer, the random number generator and the data code are mirrored in the two stacks. Both reach the same event step and the same anchor step on every seed, compared at evals 25 steps apart. The runs stop at the event, so this agreement says nothing about training after it. Bitwise equality was not expected and is not what Table 1 shows.

All 64 seeds of the tab's model fire, at a median of step 475 and a range of 400 to 700, with 0 stalls by step 3000.

Timing. On an M4 Pro in Chrome 156.0.8078.74, with nothing else on the GPU, the benchmark takes 10.3 ms per step and a median of 4.9 s to the event, with drawing off, because it holds the page still. Native PyTorch 2.15.0 is faster on the same machine and configuration. MPS takes 5.50 ms per step, about 1.9 times faster than this trainer, and 8 CPU threads take 13.93 ms. The comparison is not exact, because the native figure is a median step with no evals and with batches built ahead of time, and the 10.3 ms is wall time that includes evals. A replayed step with no evals takes 10.04 ms, and against that figure MPS is 1.83 times faster. A tab does not beat native here. It trades speed for needing nothing installed.

A watched run also draws, and its clock is a separate reading. We ran the real page with drawing on, seeds 1 to 5 over 3 rounds in headless Chrome. Its clock at the event has a median of 8.7 s and runs from 4.8 to 13.1 s. Another job held the GPU during these runs, and a tight loop on the same seed beside each one took 1.4 times its stored benchmark. That job only adds time, so each seed's fastest round is its least loaded reading, and those have a median of 5.8 s. We cannot separate the cost of drawing from that job's cost, and this is a loaded reading and not an idle one.

A replayed step splits by phase into 1.88 ms forward, 0.20 ms loss, 8.52 ms backward and 0.08 ms in the optimizer. By kernel, matrix multiplies (39 of the 65 dispatches) take 8.44 ms, or 84 percent of the step. The next largest are the embedding backward pass at 0.46 ms and token indexing at 0.26 ms, and every other kernel takes 0.20 ms or less. Chrome quantizes timestamp queries more coarsely than one dispatch, so

these come from replaying each subset of dispatches alone, the sequential-dispatch method of Maczan [12], and the kernel times sum to 10.56 ms against the whole step's 10.04. Only an M4 Pro and SwiftShader have been measured, so we make no claim about an ordinary laptop or any other GPU.

3 The instruments, and what they do not measure

These come from two sources, and we use them as Howe's code computes them. The probe is a fixed batch of repeated uniform random sequences [x; x], the setting in which Olsson et al. [2] define induction heads. The head scores are theirs. The *previous-token score* of a head is its average attention from a position to the position just before it, and the *prefix score* is its average attention to the token after the earlier copy of the current token. Olsson et al. call these previous token attention and prefix matching, and Howe [1] credits that paper with the phase change and its previous-token precursor. The rest is Howe's. *Copy advantage* is the mean cross-entropy on the first pass through x minus the mean on the second. The *event* is a copy advantage of 2 nats or more on two consecutive evals, reported at the second. The *anchor* is the first eval where a first-layer head reaches a previous-token score of 0.10. For deeper models the study takes the maximum over the first half of the applied layers, the way Howe's emergence-watch package generalizes it. The *anchor fraction* is anchor step over event step. Across 80 valid-anchor runs Howe reports a median fraction of 0.843 and freezes an envelope of event step between 1.11 and 1.36 times anchor step, which is a fraction from 0.735 to 0.901.

Copy advantage does not measure copying alone. The best possible first-pass loss is chance, $\ln 96 = 4.56$ nats, and a model that expects repeats guesses wrong on characters it has not seen, so its first pass climbs above chance. On the tab's model at the event, over 64 seeds, the first pass is 5.23, the second pass is 3.01, the advantage is 2.23, and a median 30 percent of the advantage is the first pass rising above chance. At step 3000 on seed 1 the advantage is 4.59, with a first pass of 4.73 and a second of 0.14. The event's definition stays Howe's, and it should be read as a behavioral threshold with this penalty inside it.

On Howe's language the penalty can be the whole event. Of the 206 events in run files that log both passes, 3 have a second-pass loss above chance, all on Howe's language under the clone rule (`crossed-rule-bigram` and `crossed-range-bigram`). At those evals the second pass reads 4.92 to 5.34 and the first 7.10 to 7.37, against chance at 4.56, and the top prefix score reads 0.27 to 0.50. So copy advantage crossed 2 nats there with the first pass rising, while the second pass was still worse than a uniform guess. The 28 events in `study-curriculum` did not log both passes, so we cannot say how many of them look the same. On Howe's language under the clone rule, then, the event is a gap between two passes, and it does not show a second pass better than chance.

The previous-token score has a chance level that moves with the probe. The score of uniform causal attention over T positions is $b(T) = (H_T - 1)/(T - 1)$, with H_T the harmonic number, which is 0.0349 at T=128 and 0.0594 at T=64. Howe's 0.10 is therefore 2.86 times this baseline on Howe's 128-token probe and 1.68 times it on a 64-token probe. Howe reports the baseline empirically (about 10 times baseline at 0.37). We did not find the closed form or this dependence on probe length stated within our coverage. It is a small observation, and Section 6 tests how much it explains.

The negative controls test the instruments. One layer has no head beneath it to read the previous character, and a model that never sees a repeat has nothing to copy. On the tab's model each ran 8 seeds for 6000 steps with zero events. One layer peaked at 0.913 nats of copy advantage and no repeats at 0.029. The same controls on the study model also had zero events, with peaks of 1.20 and 0.067. The one-layer peak is not zero. At that eval the first pass reads 4.80 and the second 3.89, below chance, so one layer gets part of the way and the 2-nat threshold is what separates it. Howe's event also does not show an induction head by itself. The arm in Section 7 crosses it with no prefix signature at the event.

4 One draw at Howe's config A

Config A is Howe's own setup, run on this trainer: V=2048, T=256, d=256, H=4, batch 64, lr 1e-3, Howe's bigram language, a synthetic one in which each character has a few likely successors, and Howe's clone rule, a 128-token probe, and Howe's anchor and event. Table 2 sets the result beside Howe's fleet. Seed 1 took 24

minutes at 232 ms per step. The bigram table differs from Howe's, because Howe's table cannot be rebuilt without PyTorch's generator, and Howe's own Gate C also ran a fresh table.

Table 2: Config A with Howe's anchor and event. Howe's range is in the arXiv text. The median event step and the median fraction are computed from the per-seed results in Howe's repository [3], which the paper's fleet numbers read from.

	Seeds	Event step	Anchor over event
Gunner Howe's fleet	30	median 6300, range 5650 to 7400	median 0.844
This trainer	1	6225	0.847

This is one seed, so it is one draw, inside Howe's reported range and near Howe's median fraction. It is a one-draw replication at most and it says nothing about the fleet's spread. That seed ran before the third plan was committed, under the first plan's intent. The third plan listed seeds 1 to 5 and ran none of them (Section 9). It is not a rerun of the fleet, and everything else in this paper extends Howe's event and anchor to other models and curricula.

5 What sets the onset

Earlier work asks what sets the onset on smaller or synthetic setups. Reddy [4] reproduces the data dependence of in-context learning in a two-layer attention-only network on a classification task, and traces the abrupt emergence of the induction head to the sequential learning of three nested logits. Singh et al. [5] clamp activations through training on a few-shot Omniglot task, find three interacting subcircuits behind the phase change, and show that the number of classes and the number of labels delay it through different subcircuits. Aoyama et al. [6] find that batch size and context size predict when induction heads emerge in language model pretraining, and that surface bigram repetition frequency and reliability strongly affect whether they form. That second finding is a published rationale for training on repeats, and it came before this work. In their experiment on natural text, varying the repetition rate moved the slope of the prefix score and not the emergence point, on another measure and other models, so we do not compare effect sizes with theirs.

The study model is smaller than the tab's for cost: V=96, d=64, T=64, H=4, batch 16, for 115328 parameters at two layers. Most arms ran with PyTorch on CPU on a rented 4 vCPU cloud machine, and the later ones in WGSL on the M4 Pro, which Table 1 supports. Arms ran 64 seeds unless noted and are written (layers, blocks), where a block is one set of weights and block index is layer index modulo the block count, so (8,1) is the same block applied 8 times. The main arms ran at lr 3e-3 and 1e-3, with evals every 25 steps and budgets of 6000 and 8000 steps, and runs that do not fire are right-censored at their budget. A time ratio is $\exp(\mu_a - \mu_b)$ from censored log-normal fits to two arms' event steps, with a bootstrap 95% interval from 2000 resamples over seeds. Every result below is in steps. Time per step depends on model size, and this paper compares no wall clocks across models.

5.1 Curriculum

Periodic, or tiled, means every sequence tiles a random block at a random period. Howe's clone rule copies the start of a sequence once, at a random offset. Figure 1 shows the four curricula on one model at one learning rate. Random characters made periodic reach the event at a median of step 575. Howe's bigram language made periodic takes 1025. Howe's language with the clone rule takes 7500, with 12 of 16 seeds fired by step 8000. Random characters under the clone rule fired on 0 of 16. Howe's curriculum against random characters made periodic is a time ratio of 13.1 (12.5 to 13.8), and that ratio mixes three things, because the clone arms used a repeat probability of 0.75 and periods [4, 60), and the tiled arms a probability of 1 and periods [4, 32). The third plan changed one factor at a time, with the results in Table 3.

The repeat rule is the largest single-factor contrast we can bound. On Howe's language, tiling fires on every seed and the clone rule on 2 of 16, so the clone arm has no median by step 8000 and the ratio is at least 7.80. On uniform tokens the same bound is 3.52. Two other contrasts under the clone rule, period range and language, leave one side with no median, so each has a bound of only 1.07 and may be larger, and

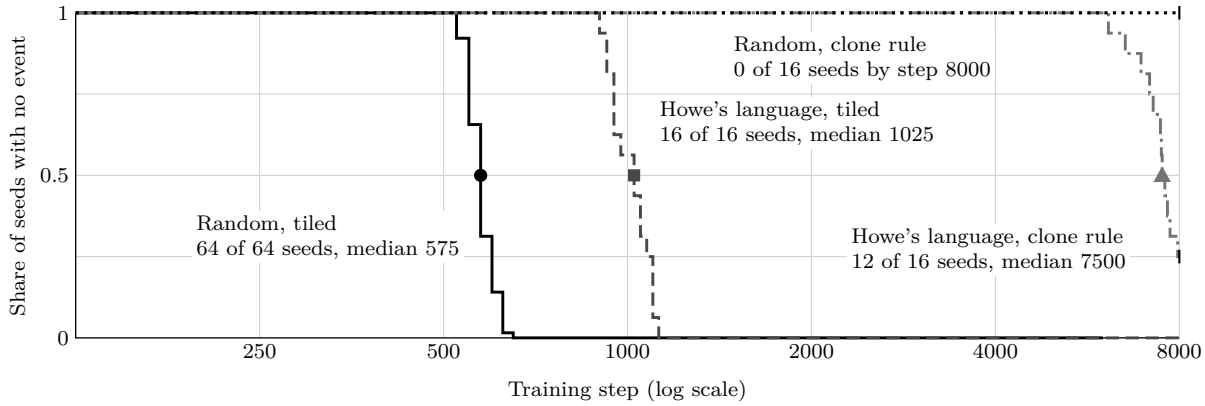


Figure 1: Kaplan-Meier survival by curriculum on the (2,2) study model at lr 3e-3. Each line is one curriculum, with 64 seeds for random tiled and 16 for each of the others, all from one budget of 8000 steps except random tiled at 6000. Censored runs end at the tick on the right: 12 of 16 fired under Howe's language with the clone rule, and 0 of 16 for random tokens with the clone rule. A marker shows each median. No tiled run was censored.

Table 3: One factor changed at a time, on the study's (2,2) model at lr 3e-3, with a budget of 8000 steps. Here p is the repeat probability, "language" is Howe's bigram language, and "uniform" is uniform random tokens. Arms have 16 seeds, except the 64-seed main arm at periods [4, 32).

Changed alone	Held fixed	One side	Other side
Repeat rule	language, p=1, [4, 32)	tiled, 16 of 16, median 1025	clone, 2 of 16
Repeat rule	uniform, p=0.75, [4, 60)	tiled, 12 of 16, median 2275	clone, 0 of 16
Period range	uniform, tiled, p=1	[4, 32), 64 of 64, median 575	[4, 60), 16 of 16, median 1025
Period range	language, clone, p=0.75	[4, 32), 1 of 16	[4, 60), 12 of 16, median 7500
Repeat rate	uniform, tiled, [4, 60)	p=1, 16 of 16, median 1025	p=0.75, 12 of 16, median 2275
Repeat rate	language, clone, [4, 32)	p=1, 2 of 16	p=0.75, 1 of 16
Language	tiled, p=1, [4, 32)	uniform, 64 of 64, median 575	language, 16 of 16, median 1025
Language	clone, p=0.75, [4, 60)	uniform, 0 of 16	language, 12 of 16, median 7500

we do not rank them. The period range changes direction between rules. Under tiling, [4, 60) against [4, 32) is a ratio of 1.78 (1.61 to 1.87), and under the clone rule the shorter periods are the worse ones. Repeat rate is smaller, at a ratio of 2.22 (1.90 to 4.12) for a probability of 0.75 against 1 under tiling, which the plan did not name as a contrast but which its arms contain. Language also changes direction. Under tiling it is a time ratio of 1.76 (1.69 to 1.82) against uniform tokens, and under the clone rule it is the only one of the two that fires.

Training on random repeats is not new. shaikabdul185-arch [19], xanderchinxyz [20] and gama [16] do it too, and Zucchet et al. [7] found that repetition can greatly speed up the emergence of sparse attention, in theory and on an in-context associative recall task. The result here is the attribution, one factor at a time, on one model. Howe's curriculum does not transfer to the tab's size in the steps a tab can spare. The tab's own model on Howe's language and clone rule fired on 0 of 8 seeds by step 8000. We stopped there, so this does not say it never would.

5.2 Weight tying

On random-repeat data, at equal depth, tying weights delayed the event. Table 4 gives both estimators at both learning rates, and Figure 2 shows the medians. The plan preregistered the censored log-normal fit, and we added the ratio of Kaplan-Meier medians after correcting the fit (Section 9). The two agree where every run fires. They disagree where a few runs stall, such as (4,1) at lr 3e-3, where 7 of 64 had no event by step 6000, because censored runs pull the fit's mean log time upward and leave the median alone. Steps are multiples of 25, so a Kaplan-Meier interval can touch its own point estimate, and these intervals are for the median on the eval grid. A percentile bootstrap on a lattice can undercover the continuous median. In

Table 4: Time ratio of a tied arm against the untied arm of equal depth, with bootstrap 95% intervals over seeds. Seeds per arm: 64 at depths 2 to 4, 32 at depth 8, 16 at depth 12.

Contrast	lr 3e-3		lr 1e-3	
	Censored log-normal	KM medians	Censored log-normal	KM medians
(2,1) / (2,2)	1.55 (1.52 to 1.59)	1.52 (1.52 to 1.57)	1.55 (1.52 to 1.57)	1.55 (1.51 to 1.59)
(3,1) / (3,3)	2.31 (2.15 to 2.54)	2.17 (2.00 to 2.21)	2.09 (2.04 to 2.14)	2.06 (2.02 to 2.15)
(4,1) / (4,4)	3.27 (2.98 to 3.70)	2.81 (2.69 to 2.96)	4.07 (3.96 to 4.19)	4.04 (3.94 to 4.25)
(4,2) / (4,4)	1.33 (1.31 to 1.36)	1.35 (1.31 to 1.35)	1.72 (1.68 to 1.76)	1.71 (1.64 to 1.77)
(8,2) / (8,8)	2.39 (2.34 to 2.45)	2.46 (2.34 to 2.46)		
(12,4) / (12,12)	1.75 (1.68 to 1.83)	1.74 (1.63 to 1.87)		

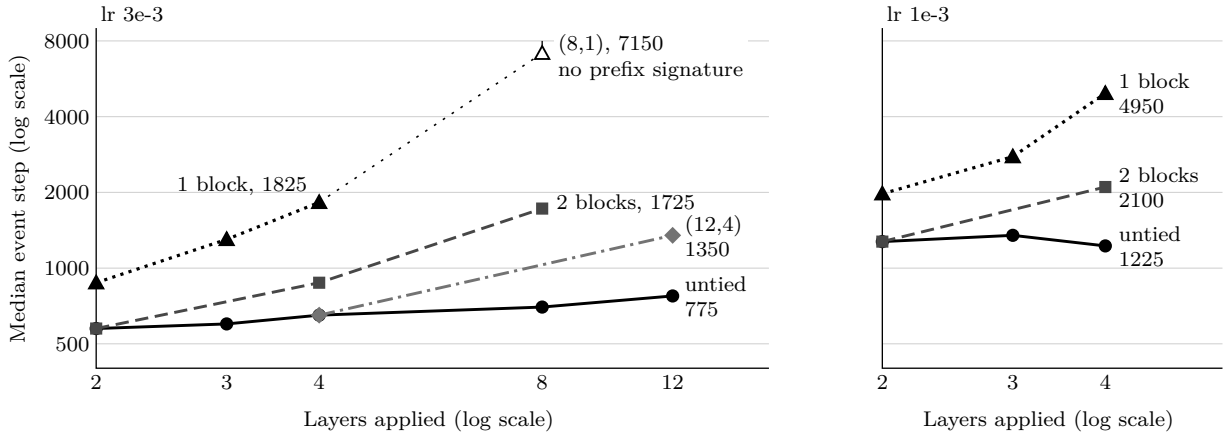


Figure 2: Median event step against depth and block count, with Kaplan-Meier 95% intervals. Lines join arms with the same block count, and the solid line joins the untied arms. Each point is one arm with 64 seeds at depths 2 to 4, 32 at (8,2) and (8,8), 16 at depth 12, and 8 at (8,1). Censored runs: 2 of 64 for (3,1) and 7 of 64 for (4,1), both at lr 3e-3 and step 6000, and no other arm shown. The open triangle is (8,1), run to step 16000, whose earlier 32 seeds had 0 events by step 4000 and are not merged into it.

a simulation at n=64, with the (2,2) arm's median and a log standard deviation of 0.051, times rounded up to the grid and 2000 draws at each position, the interval covered the grid median in 97.8 to 100 percent of draws, and the continuous median in 0.4 to 97.8 percent, depending on where that median falls inside a grid cell. We did not simulate the ratio intervals.

The design separates tying from depth. The (4,2) arm has the unique parameters of (2,2) and the depth of (4,4). Against (4,4) it is 1.33 at lr 3e-3 and 1.72 at lr 1e-3, and against (2,2) it is 1.51 and 1.66. Untied depth is nearly free in steps. Arm (4,4) against (2,2) is 1.13 at lr 3e-3 and 0.97 at lr 1e-3, and (8,8) against (4,4) is 1.09. Each step costs more, so in floating point operations (8,8) against (4,4) is 2.15.

There is no crossover at these depths. The second plan said a crossover would be claimed only if a looped arm's interval lay wholly below 1, and the lowest bound across the tying contrasts in Table 4 is 1.31. That table is not adjusted for multiple contrasts. We read 55 ratio intervals across the study and the third plan, 20 of them in the table. Read at a Bonferroni level of 0.05/55 per interval, from 40000 resamples, the lowest bound of those 20 is 1.26, so tying stays above 1 under that correction. Arm (8,1) is its own case. Its first 32 seeds had no event by step 4000, which bounds its ratio against (8,8) at no less than 5.71. The third plan then ran 8 seeds to step 16000, and 8 of 8 fire at a median of step 7150, a time ratio of 10.23 (9.67 to 10.94). We keep that arm out of the tying claim, because it shows no prefix signature at the event (Section 7).

We did not find this delay reported within our coverage. The closest work finds looped models earlier than a dense model, on other tasks, and we do not claim a contradiction. Kapl et al. [8] find in-context accuracy emerging earlier for looped models than for a 12-layer dense one, on Omniglot-based tasks. Their 4×3 loop has the shape of (12,4), so depth does not separate that result from ours. Sesodia et al. [9] report a 4×8 looped model reaching the solution sooner than an untied 32-layer model at a lower learning rate

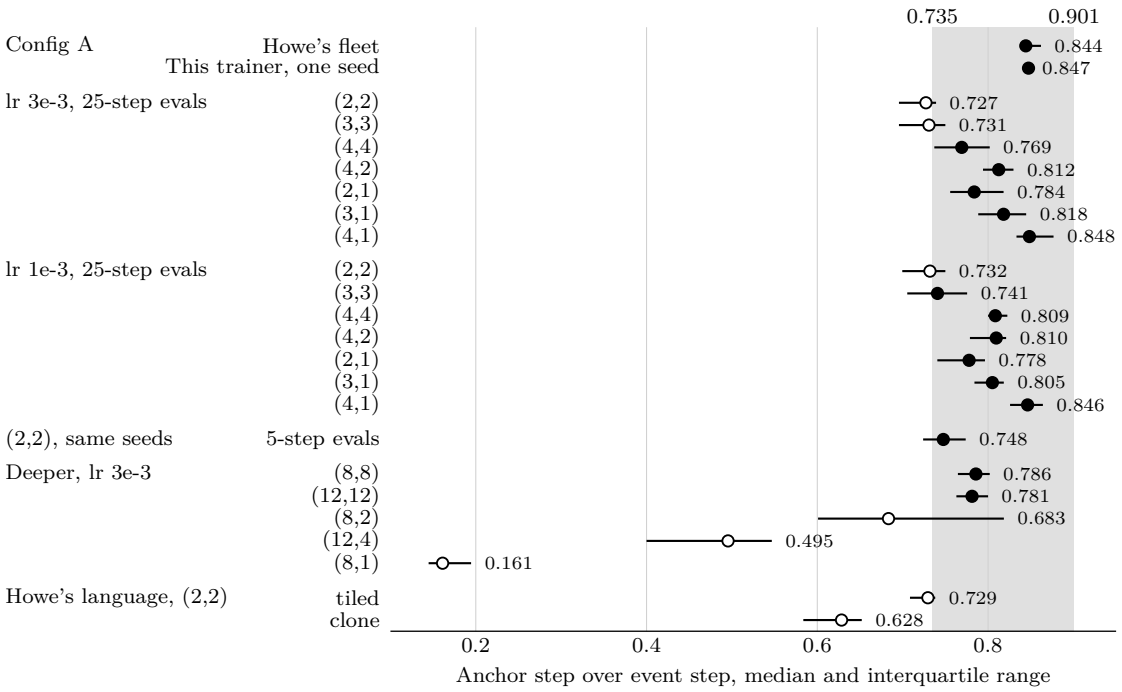


Figure 3: Anchor fraction by arm against Howe's frozen envelope (shaded). Filled markers lie inside the envelope and open markers outside. Seeds per arm: 30 for Howe's fleet, 1 for this trainer at config A, 64 at depths 2 to 4, 64 for the same-seed rerun, 32 at (8,8) and (8,2), 16 at depth 12, 8 at (8,1), and 16 for each other curriculum. Fractions count only runs where both the event and the anchor fired, so 2 censored runs of (3,1) and 7 of (4,1) at lr 3e-3, 8 anchor misses of (3,3) at lr 1e-3, and the 4 censored runs of the clone arm are not in them. Random tokens with the clone rule had no events and do not appear.

(45k training steps against 55k), on depth-bound algorithmic tasks, deeper than any arm here. The task here is copying random repeats, the models have 2 to 12 layers, and the measure is Howe's event.

We did not separate effective step size from structure. A tied block's gradient sums across its passes, Adam rescales each parameter by its running gradient magnitude, and lowering lr from 3e-3 to 1e-3 slowed looped arms further, (4,1) from 1825 to 4950. No arm scales lr with pass count. So this is a delay at these depths, learning rates and data, and we do not generalize it to tying slowing learning.

6 The forecast

Howe's law puts the anchor at a steady fraction of the time to the event. Figure 3 shows the fraction across every arm with events. Across the 14 uniform tiled arms at depths 2 to 4 the median runs from 0.727 to 0.848. The first plan's falsifier was an arm whose median falls outside Howe's envelope, and it fired for 3 arms just under it: (2,2) at lr 3e-3 (0.727), (3,3) at lr 3e-3 (0.731) and (2,2) at lr 1e-3 (0.732). The tab's own model at 25-step evals reads 0.745, just inside.

The second plan predicted that the anchor's distance from chance caused the miss. A flat 0.10 is 2.86 times the baseline on Howe's probe and 1.68 times it on ours, so it fires earlier here. The plan ran (2,2) and (2,1) at T=64 and T=128, seeds 101 to 164, at 5-step evals, under the flat anchor and under a rescaled anchor of 2.86 $b(T)$. It passes only if every arm's median under the rescaled anchor lies inside the envelope. Table 5 shows that (2,1) at T=64 reads 0.903 against 0.901, with an interval of 0.898 to 0.916, so the prediction failed literally on one arm. At T=128 both arms sit inside under either anchor.

The same table pointed at eval spacing, since the flat anchor on (2,2) at T=64 moves from 0.727 to 0.756 with finer evals, on other seeds than the study's. The third plan tested this on the study's own seeds. It reran (2,2) at lr 3e-3, seeds 1 to 64, at 5-step evals, continuing 75 steps past the event, and it voided the test unless reading those runs every 25 steps gave the study's event on every seed. As planned, the test could

Table 5: The second plan's anchor test, 64 seeds per arm. The event budget was 6000 steps, and the fractions count seeds that fired. The envelope is 0.735 to 0.901.

Arm	Events	Flat anchor, 5-step	Flat anchor, 25-step	Rescaled, 5-step
(2,2), T=64	64	0.756	0.727	0.870
(2,1), T=64	64	0.809	0.792	0.903
(2,2), T=128	59	0.798	0.792	0.797
(2,1), T=128	62	0.862	0.850	0.862

not be read on 3 seeds (17, 23 and 51), because their runs stopped before their coarse event. The other 61 reproduce the study's event and anchor. We then extended those 3 seeds to 400 steps past the event, which the plan did not name, and their curves match the first runs wherever the two overlap. With them, 64 of 64 seeds reproduce the study's event and 64 of 64 its anchor. On all 64 the fraction is 0.748 at 5-step evals against 0.727 at 25-step, the same medians as on the 61, a paired median difference of 0.026, and the share of seeds inside the envelope moves from 38 to 64 percent. So eval spacing explains part of the miss on that arm, and enough to move its median inside. The other two arms were not rerun at 5-step evals.

Spacing does not explain what is left. Howe's curriculum on the small model reads 0.628, (8,2) reads 0.683, and the deepest tied arms sit lower, (12,4) at 0.495 and (8,1) at 0.161. A 25-step grid moves the fraction of a run the length of (12,4) by about 0.019, so it cannot reach those. The untied deep arms stay inside, (8,8) at 0.786 and (12,12) at 0.781. The within-arm rank correlation of anchor and event also moves, from 0.99 at (4,1) and lr 3e-3 to 0.31 at (2,2) and -0.28 at (12,4), against Howe's 0.977 on the fleet. These gaps stay open. We report them and do not explain them.

7 Exploratory reads

These were not preregistered as tests, and they carry no explanation beyond the measurement.

No prefix signature at the event. At (8,1) every run crosses the event, with a copy advantage of 2.00 to 2.32 and a second pass of 2.89 to 3.27 at that step, and the top prefix score across all heads and layers there reads 0.024 to 0.089. Every other arm in every run file reads at least 0.27. So the event alone does not show an induction head there. These runs stop at the event, so this rests on one eval per run and says nothing about what forms later. This is 8 seeds, and we do not explain it here.

Heads in a one-block model. In (2,1) trained for 2000 steps, 2 or 3 of 4 heads score 0.10 or above as previous-token heads on pass 0 and as induction heads on pass 1, on all 8 seeds. The strongest pass 0 head scores 0.42 to 0.47, and all four heads score 0.56 to 0.66 on pass 1. This is an observation on one small model, and we claim nothing new from it. Others have looked at what tied heads do across passes at much larger scale. Kapl et al. [8] (section 4.1) find high prefix-matching heads recurring across the two applications of a shared block. Olszowiec et al. [10] (Lens 4) ask whether heads change their job across iterations, report that in looped language models they keep their roles, and note that head multiplexing has been found in looped vision transformers. Wang et al. [11] find the two visits of a repeated block attend to largely the same tokens, a top-8 overlap of 56 to 66% against 28 to 34% for a control. Those models are far larger than this one, and on a first pass here there is nothing beneath a head to match on, so we do not read this as conflicting with them.

Head placement. In (3,3) at lr 1e-3, 8 of 64 runs put the previous-token head in layer 1, with a top score of 0.217 to 0.258 there and at most 0.099 in layer 0. The anchor watches only layer 0 at that depth, so it never fired on those runs.

8 Limits

Small models. The study has width 64 and context 64, and the deepest model is 12 layers. Nothing here transfers by itself to a large model or to natural language data.

The study model is not the tab model. The tab is d=128, T=128, and the arms in Sections 5 and 6 are d=64, T=64. Every number above says which it came from.

Seeds. Most arms have 64 seeds, the single-factor arms 16, depth 8 32, depth 12 16, and (8,1) and the head read 8. All 64 seeds fire on the tab's model, which does not rule out a rare stall, and the small model does stall. At width 64 and T=128, 5 and 2 of 64 seeds had no event by step 6000.

Other GPUs. Only an M4 Pro and SwiftShader were measured. Safari, Firefox, phones and non-Apple GPUs were not. The 4.9 s comes from a benchmark that holds the page still and draws nothing. The watched figure comes from headless Chrome with another job on the GPU, and no watched run on an idle GPU is stored.

Search coverage. "Not found" means not found within the 13 search calls and direct reads logged in `research/NOVELTY-2.md`. The two closest demos are low visibility and turned up late, so others may exist.

Effective step size. Structure and step size are not separated (Section 5).

Eval spacing. Evals 25 steps apart are coarse against a 575-step onset. Only one arm was rerun at 5-step evals on its own seeds. The plan's stopping rule left 3 seeds unreadable, those whose fine and coarse events are furthest apart, and we chose the extension that reads them after seeing that.

Config A is one seed. Section 4 is one draw with a fresh bigram table, and it says nothing about the fleet's spread.

9 Preregistration and reproducibility

Three plans were committed to the project's git history, on 2026-11-01, 2026-11-01 and 2026-11-02. They are the `research/PREREG*.md` files. None was externally registered, so their dates rest on that history alone. The public history omits a private working log and copies of third-party papers and of Howe's per-seed results, with commit dates kept. The negatives and about 70% of the first plan's lr 3e-3 arms had already run, unread, when it was committed, and the plan says what had been seen. The second plan came before any of its runs. Later commits add two labeled arms to the second plan, each before its own runs, and replace gendered pronouns in the first.

Departures. The third plan's header says it came before any of its runs, and for two files that is not so, and we left its text as it was. `results/config-a.jsonl` was committed before the plan, so the plan ran zero config A seeds, and seed 1 came earlier, under the first plan's intent. `tab-native` seeds 1 to 8 were run and committed before the plan and sit unchanged in the 64. Their `tab-browser` partners were committed too, then deleted with the plan's commit and rerun under it to log both passes. The plan's spacing test could not be read as written on 3 seeds, and those were extended afterward (Section 6). The second plan's added arm, the tab's model on Howe's curriculum, ran as `tab-howe-curriculum`.

Every number here regenerates from files in the repository. Each row of a run file carries its config. The one-command reruns are `npm run study`, `npm run anchor`, `npm run compare`, `npm run compare-tab`, `npm run parity`, `npm run bench`, `npm run breakdown`, `npm run read-runs`, `python tools/analyze_third.py`, `python tools/analyze_contrasts.py` and `node tools/watched.mjs`. `node tools/runs.mjs list` names every run made on the M4 Pro, and `node tools/runs.mjs <name>` reruns one, skipping finished seeds. `paper/numbers.py` recomputes each number in this paper from the result files and fails on a mismatch, and `paper/numbers.tex`, which the text reads its numbers from, is its output.

What failed or was corrected. Four things went wrong in ways the plans or the code caught late.

The first plan's falsifier fired, on 3 arms just under Howe's envelope. We learned that an anchor fraction read at 25-step evals on a run of a few hundred steps carries a grid effect of its own, which led to the later plans.

The second plan's prediction failed. Distance from chance did not explain the first plan's miss, and the rescaled anchor missed on one arm of its own. We kept the plan's literal pass rule, even though the miss was 0.903 against 0.901.

A censored fit was silently wrong until an adversarial review of the code. The log-survival term of the likelihood clamped the survival probability at a small floor before taking its log, so the gradient for a censored

run far in the tail was zero and those runs stopped pulling on the fit, with no error raised. The first ratios at depths 3 and 4 read 2.20 and 2.83, and the corrected ones read 2.31 and 3.27. Arms with no censored runs did not move. We now compute that term with a log CDF and print the ratio of Kaplan-Meier medians beside the fit, so the two can be compared wherever runs stall.

A compound contrast was quoted as one effect. The 13.1 time ratio was first read as what the curriculum does to onset, and it mixes the repeat rule, the repeat rate and the period range. The third plan split it, and only single-factor contrasts are now credited to a single factor (Table 3). The shape of that result differs from the first reading, since the period range changes direction between rules.

Credits. Gunner Howe wrote the event, the anchor, the envelope and the transition's forecast, and Gunner Howe and Vin Howe are different people. Olsson et al. described the circuit and defined the head scores. Reddy, Singh et al. and Aoyama et al. studied what sets its formation. Code and result files are in the repository under the MIT license, and the page runs at the address under the title.

References

[1] Gunner Levi Howe. Capability emergence can be forecast: per-seed, in advance, with calibrated intervals, certified false alarms, and a blind pre-registered gate. arXiv:2609.19000, 2026.

[2] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, et al. In-context learning and induction heads. Transformer Circuits Thread, 2022. arXiv:2209.11895.

[3] Gunner Levi Howe. Research repository, subdirectory `mem-gen-delay`, file `analysis/out6/r2_scored.json`, read at commit 493c4bf. `https://github.com/gunnerhowe/Research`.

[4] Gautam Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. arXiv:2312.03002, 2023.

[5] Aaditya K. Singh, Ted Moskovitz, Felix Hill, Stephanie C. Y. Chan, and Andrew M. Saxe. What needs to go right for an induction head? A mechanistic study of in-context learning circuits and their formation. arXiv:2404.07129, 2024.

[6] Tatsuya Aoyama, Ethan Gotlieb Wilcox, and Nathan Schneider. Predicting the emergence of induction heads in language model pretraining. arXiv:2511.16893, 2025.

[7] Nicolas Zucchet, Francesco d'Angelo, Andrew K. Lampinen, and Stephanie C. Y. Chan. The emergence of sparse attention: impact of data distribution and benefits of repetition. arXiv:2505.17863, 2025.

[8] Ferdinand Kapl, Kaitlin Maile, Stefan Bauer, and Johannes von Oswald. How depth growth and looping shape in-context learning in language models. arXiv:2610.31421, 2026.

[9] Magnus Sesodia, Juil Sock, Philip Torr, and Christian Schroeder de Witt. Inside the loop: a mechanistic study of weight-tied transformers on depth-bound algorithmic tasks. arXiv:2610.24135, 2026.

[10] Pawel Olszowiec, Paulina Tomaszewska, Maciej Żelaszczyk, Michal Byra, and Neo Christopher Chung. Monitoring latent reasoning in looped language models. arXiv:2610.18127, 2026.

[11] Shaowen Wang, Ge Zhang, Kairong Luo, et al. SMELT: scaling laws for compute-matched MoE looped transformers. arXiv:2609.01343, 2026.

[12] Jędrzej Maczan. Dispatch overhead, not kernel quality: the performance bottleneck in single-stream WebGPU LLM inference. arXiv:2608.08730, 2026.

[13] Nicolas Pope and Matti Tedre. Turning language model training from black box into a sandbox. arXiv:2601.21631, 2026. Describes the Little Language Machine.

[14] Vin Howe. Sequence Toy and Piston. `https://github.com/vinhowe/piston`, `https://sequence.toys`.

[15] TheoremPath. Induction Heads Lab. `https://theorempath.com/demo/induction-heads-lab`.

[16] pariharshyamu. gama. `https://github.com/pariharshyamu/gama`.

[17] jbingen. TabGPT. `https://github.com/jbingen/tabgpt`.

[18] toprakdeviren. webgpu-llm. `https://github.com/toprakdeviren/webgpu-llm`.

[19] shaikabdul185-arch. induction-heads-from-scratch. `https://github.com/shaikabdul185-arch/induction-heads-from-scratch`.

[20] xanderchinxyz. forming-induction-heads. https://github.com/xanderchinxyz/forming-induction-heads.